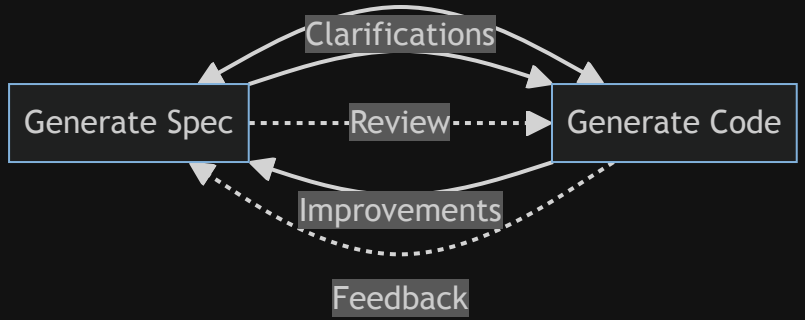
OpenSpec & Copilot: Accelerating Change Management in Brownfield Projects

> A practical guide to using Spec-driven Development (SDD) and AI agents for safe, traceable, and efficient evolution of complex codebases.

by Sathishkumar Deena Kirupakaran | Mar 2026

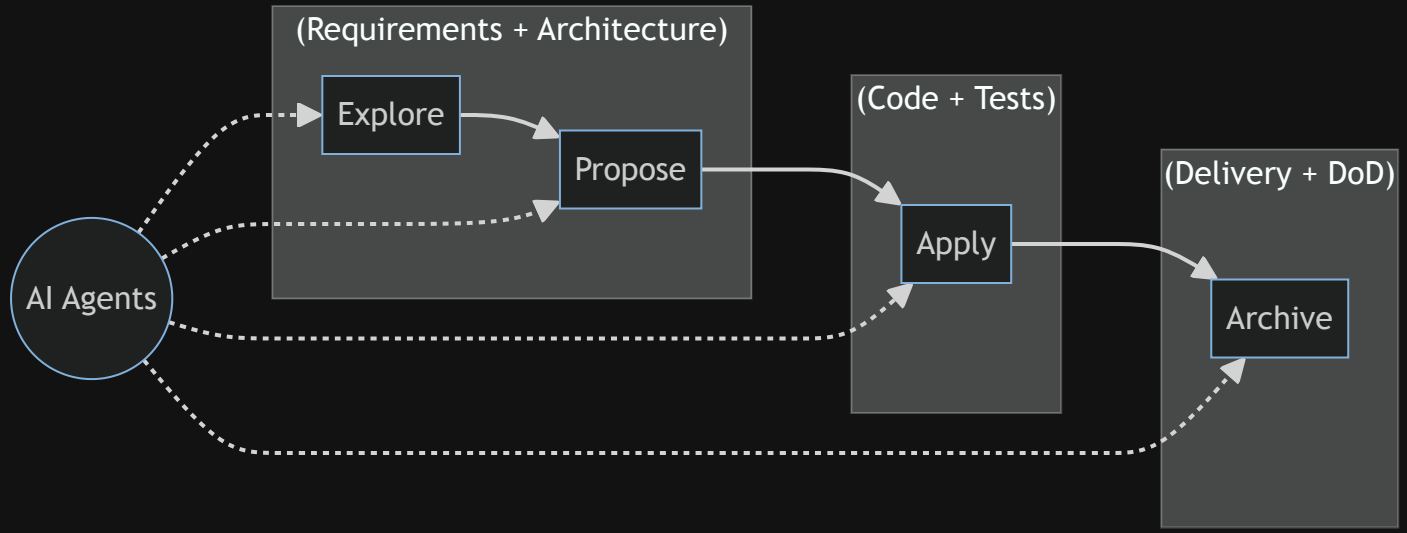
What is Spec-driven Development? (SDD)



- Ensures code and requirements stay in sync for fewer bugs.
- Makes changes traceable and collaboration easier.

Openspec: Spec-based AI Management Tool

Note: OpenSpec Spec-driven development (SDD) empowers AI coding assistants to accelerate every phase—exploring, proposing, applying, and archiving changes—by providing clear, modular specs and traceable workflows.



https://github.com/Fission-AI/OpenSpec

Usecase: Firebolt C++ Transport

https://github.com/rdkcentral/firebolt-cpp-transport

Why this repo?

- Brownfield codebase with unique challenges that require careful change management and traceability.
- More than 90% of our org have Brownfield codebases.
- New functionality handed off from another org lacking requirements and documentation.

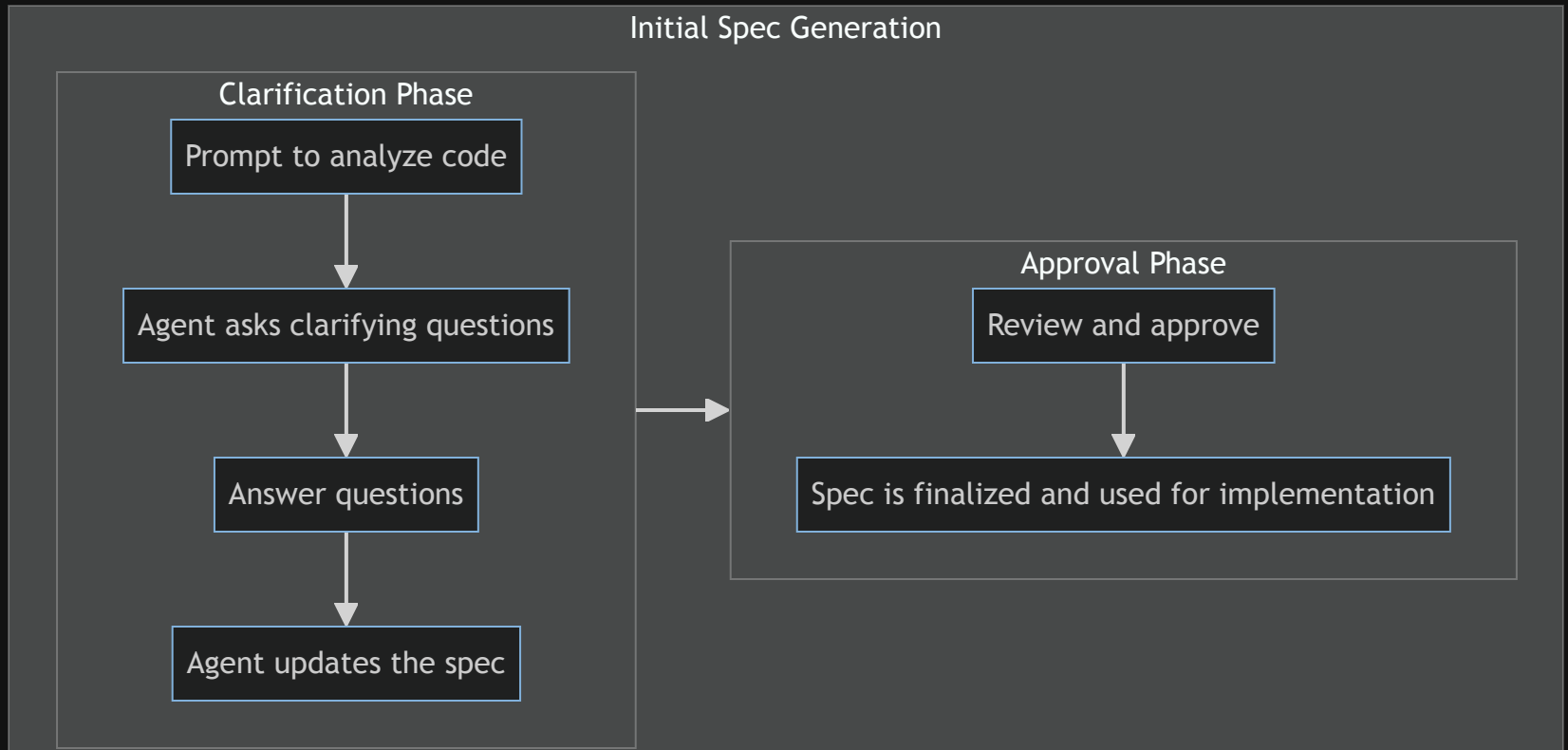
What is the objective?

- Explore and create specs for existing code.
- Analyze missing functionality.
- Propose and implement changes.
- Document and share learnings.

> Use Github Copilot and preferrably a simple Agent like GPT-4.1 to assist with all phases of the process, from spec generation to code implementation and documentation.

[Explore] Interactive Spec Generation

```
$ /ospx-explore "Explore current repository and generate specs for transport layer. Prompt questions
when you need more info" --output "specs/"
```



[Explore]: Generated Specs

```
$ ls
specs/
├── cpp_specifics_spec.md
├── header_interfaces_spec.md
├── json_rpc_handling_spec.md
├── transport_layer_spec.md
├── transport_recommendations_spec.md
changes/
├── archive/
```

- Challenge: Specs were monolithic and hard to update
- Approach: Broke down specs into modular components
- Result: Easier to review, update, and track changes

[Explore] Generate Recommendations

- Support headers in the transport layer
- Add robust support for JSON-RPC batch requests
- Make retry logic optional and client-driven
- Ensure full JSON-RPC compliance (batch, error reporting)
- Provide async batch handling and granular error reporting
- Document thread safety, callback registration, and error handling

[Propose] Add Header Support

```
$ /ospx-propose "Add header support to transport layer" --spec "header_support_spec.md" --tasks "tasks.md"
 ...
Refine Proposal
 ...
$ ls # After proposal finalization
└── add-header-support/
    ├── add_header_support_proposal.md   # Captures the requirement and why this change is needed
    ├── design.md                        # Explains how the change can be implemented
    ├── specs/
    │   └── header_support_spec.md       # Details the expected behavior and API for the change
    └── tasks.md                         # Lists the concrete steps needed to complete the change
```

[Apply] Implementing Tasks with Copilot

```
$ /ospx-apply "add-header-support"
```

- ☑Update `Config` struct to include `headers` field
- ☑Update `connect` method to accept and use headers
- ☑Implement header injection in transport layer
- ☑Implement response header retrieval in transport layer
- ☑Expose `getResponseHeader` in `IGateway` interface
- ☑Ensure thread safety and error handling for header operations
- ☑Add unit and integration tests for header injection and retrieval
- ☑Update documentation for new header support
- ☐Review and archive the change in OpenSpec

[Archive] Documenting and Sharing Learnings

```
$ /ospx-archive "add-header-support"
```

Update tasks.md - [x] Review and archive the change in OpenSpec

- Captures the rationale, design decisions, implementation details, and results of the change.
- Serves as a reference for future changes and promotes knowledge sharing across the team.

Key Takeaways

Openspec Branch https://github.com/rdkcentral/firebolt-cpp-transport/tree/feature/openspec

- AI accelerates structured change management
- OpenSpec ensures traceability and review
- Copilot improves productivity and documentation
- Openspec content also powers Slidev for presentations

Next Steps

- Expand OpenSpec usage
- Share learnings with the team
- Continue integrating AI into other repositories and workflows

Q&A

- Questions and discussion

Thank you